

فاز چهارم

کیوان ثاقب مجیدزاده

موضوع برنامه

هدف این برنامه پیدا کردن یک زیرمجموعه کمینه از مقدمات یک استنتاج معتبر است. اگر مجموعه مقدمات را با Γ و نتیجه را با ψ نشان دهیم، ورودی تضمین می کند که:

$$\Gamma \models \psi.$$

در بسیاری از موارد، تمام مقدمات برای نتیجه گیری لازم نیستند. برنامه با روش Deletion-Based مقدمات غیرضروری را حذف می کند و شماره مقدماتی را برمی گرداند که در زیرمجموعه نهایی باقی مانده اند. اگر خود مقدمات ناسازگار باشند، برنامه به جای کمینه سازی استنتاج نسبت به نتیجه، یک MUS پیدا می کند. در هر دو حالت، حذف هر عضو باقی مانده ویژگی مورد نظر را از بین می برد.

تعریف زیرمجموعه کمینه

فرض می کنیم:

$$\Gamma = \{\varphi_1, \varphi_2, \dots, \varphi_n\}.$$

زیرمجموعه $\Gamma' \subseteq \Gamma$ برای نتیجه ψ کمینه است اگر دو شرط زیر برقرار باشند:

$$\Gamma' \models \psi,$$

و برای هر $\varphi_i \in \Gamma'$ داشته باشیم:

$$\Gamma' \setminus \{\varphi_i\} \not\models \psi.$$

بنابراین هیچ مقدمه ای را نمی توان از Γ' حذف کرد و همچنان استنتاج را برقرار نگه داشت.

تفاوت Minimum و Minimal

هدف برنامه پیدا کردن یک زیرمجموعه Minimal است، نه الزاماً Minimum. در حالت Minimal، حذف هر عضو باقی مانده ویژگی مورد نظر را از بین می برد. در حالت Minimum، تعداد اعضای زیرمجموعه باید در میان تمام جواب های ممکن کمترین باشد.

برای مثال، اگر مقدمات و نتیجه به صورت زیر باشند:

$$\Gamma = \{q, p, p \rightarrow q\}, \quad \psi = q,$$

هر دو مجموعه زیر Minimal هستند:

$$\{q\}, \quad \{p, p \rightarrow q\}.$$

اما فقط مجموعه $\{q\}$ از نظر تعداد اعضا Minimum است. الگوریتم این برنامه از نوع Deletion-Based است و تضمین نمی‌کند که جواب دارای کمترین تعداد عضو باشد. برای بیان رسمی معیار کمترین تعداد عضو، اصطلاح Minimum Cardinality به کار می‌رود.

حالت ناسازگاری مقدمات

اگر هیچ انتسابی نتواند همه مقدمات را هم‌زمان درست کند، مجموعه Γ ناسازگار است:

$$\bigwedge_{\varphi \in \Gamma} \varphi \equiv F.$$

در این حالت، برنامه یک MUS پیدا می‌کند. نام کامل MUS، عبارت Minimal Unsatisfiable Subset است. مجموعه Γ' یک MUS است اگر:

$$\bigwedge_{\varphi \in \Gamma'} \varphi \equiv F,$$

اما با حذف هر عضو، مجموعه باقی‌مانده سازگار شود. در منطق کلاسیک، از مجموعه ناسازگار هر نتیجه‌ای به دست می‌آید؛ بنابراین استنتاج ورودی همچنان معتبر است، اما برنامه باید ناسازگاری مقدمات را پیش از کمینه‌سازی نسبت به نتیجه تشخیص دهد.

نمادهای قابل قبول

نماد متنی	معنی
a تا z	متغیر گزاره‌ای تک حرفی
T	ثابت درست
F	ثابت نادرست
$\neg A$	نقیض A
$(A \& B)$	عطف $A \wedge B$
$(A B)$	فصل $A \vee B$
$(A \rightarrow B)$	شرطی $A \rightarrow B$
$(A \leftrightarrow B)$	دوشرطی $A \leftrightarrow B$

متغیرهای گزاره‌ای فقط حروف کوچک انگلیسی هستند. هر زیرفرمول دوتایی باید داخل یک جفت پرانتز نوشته شود. نقیض مستقیماً به زیرفرمول بعدی متصل می‌شود و پرانتز جداگانه نمی‌گیرد. فرمول‌ها نیز نباید فاصله اضافی داشته باشند.

قالب ورودی

خط اول عدد n ، یعنی تعداد مقدمات، است:

$$1 \leq n \leq 15.$$

در n خط بعدی، مقدمات به ترتیب ورودی قرار دارند و آخرین خط شامل فرمول نتیجه است:

```
n
premise_1
premise_2
...
premise_n
conclusion
```

شماره هر مقدمه در خروجی یک مبنا است؛ یعنی مقدمه اول شماره یک دارد.

قالب خروجی

اگر مجموعه مقدمات سازگار باشد، خط اول خروجی دقیقاً عبارت زیر است:

Minimal Subset

خط دوم شامل شماره مقدمات باقی مانده در زیرمجموعه کمینه است.

اگر مجموعه مقدمات ناسازگار باشد، خط اول خروجی عبارت زیر است:

Inconsistent

خط دوم شامل شماره مقدمات عضو MUS است. شماره‌ها در هر دو حالت به ترتیب صعودی و با یک فاصله چاپ می‌شوند.

ایده اصلی استفاده از bitset

اگر در تمام مقدمات و نتیجه، k متغیر متمایز وجود داشته باشد، تعداد تمام انتساب‌های ممکن برابر است با:

$$V = 2^k.$$

برنامه نتیجه هر فرمول را در تمام انتساب‌ها با یک عدد صحیح دارای V بیت نمایش می‌دهد. مقدار هر بیت مطابق جدول زیر است:

bit_i	وضعیت فرمول در انتساب شماره i
1	درست
0	نادرست

متغیر `full_mask` عددی است که تمام V بیت معتبر آن یک هستند:

$$\text{full_mask} = 2^V - 1.$$

عطف چند مقدمه با عملگر بیتی AND محاسبه می‌شود. هر بیت یک در نتیجه نشان می‌دهد که تمام مقدمات فعال در انتساب متناظر درست هستند.

ساختار کلی اجرای برنامه

مراحل اجرای برنامه به ترتیب زیر است:

۱. خواندن تعداد مقدمات، مقدمات و نتیجه؛
۲. استخراج متغیرهای متمایز از تمام فرمول‌ها؛
۳. ساخت `bit mask` متغیرها و `full_mask`؛
۴. تجزیه و ارزیابی هر مقدمه و نتیجه؛
۵. محاسبه عطف تمام مقدمات؛
۶. تشخیص سازگاری یا ناسازگار بودن مجموعه کامل مقدمات؛
۷. اجرای الگوریتم Deletion-Based متناسب با حالت تشخیص داده‌شده؛
۸. تبدیل شماره‌های صفرمینا به یک‌مینا و چاپ آن‌ها.

کلاس FormulaParser

کلاس FormulaParser فرمول را از چپ به راست تجزیه می کند و همزمان نتیجه آن را در تمام انتساب ها به صورت bitset محاسبه می کند. هر فراخوانی parse_formula یک عدد صحیح برمی گرداند که نتیجه زیرفرمول فعلی است.

ویژگی های کلاس عبارت اند از:

ویژگی	توضیح
text	متن فرمول ورودی
variable_masks	نگاشت هر متغیر به bit mask آن
full_mask	عددی که تمام بیت های معتبر آن یک هستند
position	محل فعلی خواندن متن فرمول

متد parse پس از محاسبه نتیجه بررسی می کند که همه کاراکترهای ورودی مصرف شده باشند. اگر کاراکتر اضافی باقی مانده باشد، خطای ValueError ایجاد می شود.

تجزیه و ارزیابی فرمول

متد parse_formula حالت های زیر را پردازش می کند:

- متغیر: bit mask همان متغیر برگردانده می شود.
- ثابت T: مقدار full_mask برگردانده می شود.
- ثابت F: مقدار صفر برگردانده می شود.
- نقیض: نتیجه زیرفرمول بعدی در محدوده full_mask معکوس می شود.
- فرمول دوتایی: طرف چپ، عملگر و طرف راست خوانده و نتیجه محاسبه می شود.

نقیض با رابطه زیر محاسبه می شود:

$$\neg A = \text{full_mask} \wedge A.$$

متد parse_operator ابتدا <->، سپس >- و بعد & و | را بررسی می کند تا عملگر بلندتر به اشتباه به عنوان بخشی از عملگر کوتاه تر خوانده نشود.

اجرای عملگرهای منطقی

عملیات بیتی	رابطه منطقی	عملگر
left & right	$A \wedge B$	&
left right	$A \vee B$	
(full_mask ^ left) right	$\neg A \vee B$	->
full_mask ^ (left ^ right)	$\neg(A \oplus B)$	<->

عملگرهای بیتی باعث می‌شوند مقدار فرمول در تعداد زیادی انتساب به صورت هم‌زمان محاسبه شود.

توابع ساخت ماسک متغیرها

تابع `collect_variables` همه حروف کوچک ظاهرشده در مقدمات و نتیجه را در یک set قرار می‌دهد و آن‌ها را به ترتیب الفبایی برمی‌گرداند.

تابع `build_variable_masks` برای هر متغیر یک bit mask می‌سازد. اگر تعداد متغیرها k باشد:

$$\text{valuation_count} = 2^k,$$

$$\text{full_mask} = 2^{\text{valuation_count}} - 1.$$

برای متغیر قرارگرفته در موقعیت i ، اندازه بلوک‌های متوالی `True` و `False` برابر است با:

$$\text{block_size} = 2^{k-i-1}.$$

سپس بلوک بیت‌های یک در تمام محل‌هایی قرار می‌گیرد که مقدار آن متغیر `True` است.

تابع `evaluate_formula`

این تابع یک نمونه از `FormulaParser` می‌سازد و متد `parse` را فراخوانی می‌کند. خروجی، یک عدد صحیح `bitset` است که مقدار فرمول را در تمام انتساب‌ها ذخیره می‌کند.

تابع `calculate_conjunction`

این تابع عطف تمام مقدمات فعال را محاسبه می‌کند. ورودی‌های آن عبارت‌اند از:

- `premise_masks`: ماسک نتیجه هر مقدمه؛
- `active`: فهرستی از مقادیر `True` و `False` که فعال بودن هر مقدمه را مشخص می‌کند؛
- `full_mask`: مقدار اولیه عطف.

متغیر `result` ابتدا برابر `full_mask` است. سپس برای هر مقدمه فعال داریم:

```
result &= premise_mask
```

اگر `result` صفر شود، تابع حلقه را متوقف می‌کند؛ زیرا عطف صفر با هر ماسک دیگری همچنان صفر باقی می‌ماند. اگر نتیجه نهایی صفر باشد، هیچ انتسابی تمام مقدمات فعال را درست نمی‌کند و این مجموعه ناسازگار است.

تابع `entails`

این تابع بررسی می‌کند که آیا عطف مقدمات فعال، نتیجه را به دنبال دارد یا خیر. ابتدا ماسک انتساب‌هایی ساخته می‌شود که نتیجه در آن‌ها نادرست است:

```
false_conclusion_mask = full_mask ^ conclusion_mask.
```

سپس مثال‌های نقض احتمالی محاسبه می‌شوند:

```
counterexamples = premise_conjunction & false_conclusion_mask.
```

اگر مقدار `counterexamples` صفر باشد، هیچ انتسابی وجود ندارد که همه مقدمات را درست و نتیجه را نادرست کند؛ بنابراین استنتاج برقرار است.

الگوریتم `Deletion-Based` برای حالت سازگار

تابع `find_minimal_entailing_subset` از مجموعه‌ای شروع می‌کند که تمام مقدمات در آن فعال هستند:

```
active = [True] * premise_count
```

مقدمات به ترتیب ورودی بررسی می‌شوند. برای مقدمه شماره i مراحل زیر انجام می‌شود:

۱. مقدمه به صورت موقت غیرفعال می‌شود.

۲. عطف مقدمات فعال دوباره محاسبه می‌شود.

۳. اگر مقدمات باقی‌مانده همچنان نتیجه را به دنبال داشته باشند، حذف مقدمه دائمی می‌شود.

۴. در غیر این صورت، مقدمه دوباره فعال می‌شود.

پس از پایان حلقه، حذف هر مقدمه باقی‌مانده باعث از بین رفتن استنتاج می‌شود. بنابراین مجموعه نهایی Minimal است. ممکن است ترتیب متفاوت بررسی مقدمات، زیرمجموعه Minimal متفاوتی تولید کند. این کد دقیقاً ترتیب ورودی را استفاده می‌کند.

الگوریتم Deletion-Based برای حالت ناسازگار

تابع `find_minimal_unsatisfiable_subset` نیز با فعال‌بودن تمام مقدمات شروع می‌کند. هر مقدمه موقتاً حذف و عطف مجموعه باقی‌مانده محاسبه می‌شود:

- اگر عطف همچنان صفر باشد، مجموعه هنوز ناسازگار است و حذف مقدمه دائمی می‌شود.

- اگر عطف غیرصفر شود، مجموعه سازگار شده است و مقدمه باید دوباره فعال شود.

در پایان، مجموعه باقی‌مانده یک MUS است؛ زیرا خود مجموعه ناسازگار است، اما حذف هر عضو آن، مجموعه را سازگار می‌کند.

چرا یک بار بررسی هر مقدمه کافی است؟

در حالت سازگار، اگر حذف یک مقدمه باعث شکست استنتاج شود، حذف مقدمات بیشتر نمی‌تواند آن استنتاج را دوباره برقرار کند؛ زیرا با حذف مقدمه، شرط درست‌بودن مقدمات ضعیف‌تر و تعداد مثال‌های نقض احتمالی بیشتر می‌شود.

در حالت ناسازگار نیز اگر حذف یک مقدمه مجموعه را سازگار کند، حذف مقدمات بیشتر نمی‌تواند آن را دوباره ناسازگار کند. بنابراین نیازی نیست مقدمات حفظ‌شده در دورهای قبلی دوباره بررسی شوند.

تبدیل شماره‌ها به یک مبنا

فهرست `active` از اندیس صفرمبنا استفاده می‌کند، اما شماره خروجی باید یک‌مبنا باشد. به همین دلیل، هنگام ساخت جواب یک واحد به هر اندیس فعال اضافه می‌شود:

```
return [
    index + 1
    for index, is_active in enumerate(active)
    if is_active
]
```

چون پیمایش `enumerate` از ابتدا به انتها انجام می‌شود، شماره‌های خروجی به‌طور طبیعی صعودی هستند.

تابع solve

تابع solve ابتدا خط‌های خالی را حذف و تعداد مقدمات را از خط اول می‌خواند. تعداد خط‌های مورد انتظار برابر است با:

$$\text{expected_line_count} = n + 2.$$

اگر تعداد خط‌ها متفاوت باشد، خطای ValueError ایجاد می‌شود. سپس مقدمات، نتیجه و اجتماع متغیرهای آن‌ها استخراج می‌شوند. هر مقدمه به یک bitset تبدیل می‌شود و ماسک نتیجه نیز جداگانه محاسبه می‌شود. فهرست all_active تمام مقدمات را فعال نگه می‌دارد و full_premise_conjunction عطف مجموعه کامل مقدمات است. اگر این مقدار صفر باشد، تابع find_minimal_unsatisfiable_subset اجرا و خروجی Inconsistent تولید می‌شود. اگر مقدار غیرصفر باشد، مقدمات سازگار هستند. در این حالت تابع find_minimal_entailing_subset اجرا و خروجی Minimal Subset تولید می‌شود. کد در این بخش به تضمین صورت سؤال متکی است که استنتاج کامل از ابتدا معتبر است.

تابع main

تابع main ابتدا تعداد مقدمات را می‌خواند، سپس دقیقاً به همان تعداد مقدمه و در پایان یک نتیجه دریافت می‌کند:

```
premise_count = int(input().strip())
premises = [
    input().strip()
    for _ in range(premise_count)
]
conclusion = input().strip()
```

خط‌های ورودی با join و "\n" به یک رشته تبدیل و به solve داده می‌شوند. در پایان، خروجی solve چاپ می‌شود.

اجرای نمونه اول: مقدمات سازگار

ورودی:

3
p
(p|q)
q
(p&q)

مقدمه دوم برای نتیجه‌گیری لازم نیست؛ زیرا مقدمات اول و سوم مستقیماً $p \wedge q$ را برقرار می‌کنند. الگوریتم با حذف آزمایشی مقدمات، شماره‌های یک و سه را نگه می‌دارد.
خروجی:

Minimal Subset
1 3

اجرای نمونه دوم: مقدمات ناسازگار

ورودی:

2
p
!p
q

دو مقدمه p و $\neg p$ نمی‌توانند در هیچ انتسابی هم‌زمان درست باشند. مجموعه مقدمات ناسازگار است و هر دو مقدمه برای حفظ ناسازگاری لازم هستند.
خروجی:

Inconsistent
1 2

پیچیدگی زمانی و حافظه

اگر تعداد متغیرهای متمایز را k و تعداد انتساب‌ها را $V = 2^k$ در نظر بگیریم، هر ماسک دارای V بیت است. رشد تعداد انتساب‌ها نسبت به k نمایی است.

اگر تعداد مقدمات را n در نظر بگیریم، در هر مرحله Deletion-Based ممکن است عطف حداکثر n ماسک دوباره محاسبه شود. چون حداکثر n مرحله حذف وجود دارد، تعداد ترکیب ماسک‌ها در بدترین حالت از مرتبه زیر است:

$$O(n^2).$$

هر عملیات بیتی روی عددی با V بیت اجرا می‌شود. بنابراین هزینه واقعی علاوه بر n^2 به اندازه ماسک‌ها نیز وابسته است. حافظه اصلی برای نگهداری ماسک متغیرها و مقدمات مصرف می‌شود.

محدودیت‌های برنامه

- تعداد مقدمات باید حداقل یک باشد.
- استنتاج کامل باید مطابق تضمین ورودی معتبر باشد.
- نام هر متغیر فقط یک حرف کوچک انگلیسی از a تا z است.
- زیرفرمول‌های دوتایی باید به‌طور کامل داخل پرانتز قرار گیرند.
- فرمول‌ها نباید فاصله اضافی داشته باشند.
- خروجی Minimal است و الزاماً Minimum نیست.
- تعداد انتساب‌ها با تعداد متغیرها به صورت نمایی افزایش می‌یابد.
- افزایش حد بازگشت به 10000 امکان پردازش فرمول‌های عمیق‌تر را فراهم می‌کند، اما ورودی بسیار عمیق همچنان می‌تواند منابع زیادی مصرف کند.